

Lesson 2: JSX, Props, and Component Trees

Mission Brief

Mission Control needs more than fixed text — it needs to display *different* crew member cards, each with its own name, callsign, and status, without copy-pasting the same component three times with tiny edits. Today you learn **JSX** properly and **props**, the mechanism that lets one component definition render many different-looking instances, just like a function takes different arguments.

Learning Objectives

By the end of this lesson, you will be able to:

- Read and write JSX confidently, including embedding JavaScript expressions with `{ }`
- Explain how JSX compiles down to `React.createElement()` calls
- Pass data into a component using props
- Destructure props in a component's function signature
- Render a list of components dynamically from an array, using `key`

What You'll Build

A `CrewRoster` feature: a `CrewCard` component that accepts `name`, `callsign`, and `status` as props, rendered multiple times from a real array of crew data using `.map()`.

Prerequisites

- Lesson 1 (components, project setup)
- Comfort with JavaScript arrays and `.map()`

Key Concepts

- JSX syntax rules (single root, `className` not `class`, self-closing tags)
- Embedding JavaScript expressions in JSX with `{ }`
- Props as function parameters
- Destructuring props
- Rendering lists with `.map()`
- The `key` prop and why it matters

Concept Explanation

JSX is a syntax extension that lets you write HTML-like markup directly inside JavaScript. It looks like HTML, but it isn't — it's *syntactic sugar* that gets compiled (by Vite, behind the scenes) into plain JavaScript function calls, roughly `React.createElement("h1", null, "Hello")`. Because JSX is just JavaScript in disguise, you can drop back into real JavaScript anywhere by wrapping an expression in curly braces `{ }` — `<h1>{missionName}</h1>` embeds the value of the `missionName` variable directly into the rendered text. Only *expressions* are allowed inside `{ }` (things that produce a value), not statements like `if` or `for` loops — this is a common point of confusion for beginners.

JSX has a few rules that differ from HTML: because `class` is a reserved word in JavaScript, JSX uses `className` instead for CSS classes. Every tag must be properly closed, including "void" elements like `<img />` and `<input />` that don't need a closing tag in HTML but do in JSX. And, as covered in Lesson 1, a component must return exactly one root element (or an empty `<>...</>` Fragment wrapping multiple children).

Props ("properties") are how data flows *into* a component from its parent, exactly like arguments flow into a function. When you write `<CrewCard name="Nova" status="On Duty" />`, React passes an object `{ name: "Nova", status: "On Duty" }` as the single argument to the `CrewCard` function. Inside the component, you typically **destructure** that object right in the function signature — `function CrewCard({ name, status }) { ... }` — so you can refer to `name` and `status` directly instead of writing `props.name` everywhere. Props are **read-only** from the component receiving them: a component should never reassign its own props; if something needs to change over time, that's a job for *state*, which you'll meet in Lesson 3.

Props become especially powerful combined with rendering **lists**. If you have an array of crew objects, you don't write one `<CrewCard>` per person by hand — you call `.map()` on the array to transform each data object into a `<CrewCard>` element, producing an array of JSX elements that React renders in order. Whenever you render a list this way, React requires a special `key` prop on each item — a unique, stable identifier (often an `id` from your data) that React uses internally to track which item is which across re-renders, so it can correctly reorder, add, or remove items without mixing up their internal state. Using an array *index* as the key works but is discouraged when the list can be reordered or filtered, because indexes shift while your actual data doesn't.

Guided Coding

```
// src/data/crew.js

// A plain array of data – no JSX here, just JavaScript objects. This is the
// same pattern you'll use for real product/post data in the portfolio
// projects later in this course.
export const crew = [
  { id: "crew-1", name: "Nova Reyes", callsign: "Pathfinder", status: "On Duty" },
  { id: "crew-2", name: "Kai Thompson", callsign: "Stargazer", status: "Resting" },
  { id: "crew-3", name: "Priya Anand", callsign: "Voyager", status: "On Duty" },
];
```

```
// src/components/CrewCard.jsx

// Destructuring props directly in the function signature. `name`,
// `callsign`, and `status` are read from whatever object gets passed in
// as props by the parent component.
export default function CrewCard({ name, callsign, status }) {
  const isOnDuty = status === "On Duty";

  return (
    <div className="crew-card">
      <h3>{name}</h3>
      <p>Callsign: {callsign}</p>
      /* A JavaScript expression inside {} – here a ternary, since JSX
         can't contain a plain if-statement */
      <p>Status: {isOnDuty ? "🟢 On Duty" : "😴 Resting"}</p>
    </div>
  );
}
```

```
// src/components/CrewRoster.jsx

import CrewCard from "../CrewCard";
import { crew } from "../data/crew";

export default function CrewRoster() {
  return (
    <section>
      <h2>Crew Roster ({crew.length} members)</h2>
      <div className="roster-grid">
        {crew.map((member) => (
          // key is required whenever you render a list – it should be a
          // stable, unique value from the actual data (member.id), not the
          // array index, so React can track each card correctly.
          <CrewCard
            key={member.id}
            name={member.name}
            callsign={member.callsign}
            status={member.status}
          />
        ))}
      </div>
    </section>
  );
}
```

```
// src/App.jsx
import CrewRoster from "../components/CrewRoster";

export default function App() {
  return (
    <div>
      <h1>Lumexa Mission Control</h1>
      <CrewRoster />
    </div>
  );
}
```

Code Walkthrough

- `crew.js` : a plain array of objects, completely separate from any component. Keeping data separate from UI components is a habit that pays off enormously as apps grow — it's exactly the shape you'll reuse for `product.js` and `posts.js` in the portfolio projects.
- `CrewCard({ name, callsign, status })` : destructuring pulls the three props straight out of the incoming props object, so the rest of the function can just say `name` instead of `props.name`.
- `{isOnDuty ? "🟢 On Duty" : "😴 Resting"}` : a ternary expression is the idiomatic way to choose between two small pieces of JSX/text inline, since a full `if` statement isn't valid directly inside `{ }`.
- `crew.map((member) => (...))` : transforms the array of data into an array of `<CrewCard>` elements — one per crew member — which React then renders in the returned position.
- `key={member.id}` : uses the crew member's real, stable `id` from the data rather than the array index, so React can correctly match each rendered card to its data even if the array is later reordered or filtered.

Student Mission

1. Add a fourth crew member to the `crew` array in `crew.js`, and confirm a new `CrewCard` appears automatically — without touching `CrewRoster.jsx` at all.
2. Add a `role` field to each crew member (e.g. `"Pilot"`, `"Engineer"`), pass it as a new prop to `CrewCard`, and display it.
3. Change `CrewCard`'s status ternary so status `"Off Duty"` shows a third message (`"🔴 Off Duty"`) — you'll need something other than a plain ternary here (a series of `if` statements inside the function body before the `return`, or a lookup object, both work).

Challenge

Refactor `CrewCard` to accept a single `member` prop (the whole object) instead of three separate props, i.e. `<CrewCard key={member.id} member={member} />`, and destructure `{ name, callsign, status }` from

`member` *inside* the function body. This "pass the whole object" pattern is extremely common in real React code once a component's data starts growing more fields.

Experiment/Extension

In `CrewRoster.jsx`, temporarily change `key={member.id}` to `key={Math.random()}` and reload the page a few times. Notice each card fully re-mounts (any internal state would reset) instead of React recognizing "this is the same card as before." This demonstrates concretely why keys must be *stable* across renders, not regenerated every time.

Common Mistakes

- Writing `class="crew-card"` instead of `className="crew-card"` — React will render a console warning and the class won't reliably apply through JSX's class handling.
- Forgetting the `key` prop when rendering a list with `.map()`, which produces a console warning: "Warning: Each child in a list should have a unique 'key' prop."
- Using the array index as `key` on a list that can be reordered, filtered, or have items removed from the middle — this can cause visually wrong or buggy re-renders, especially when list items hold their own internal state.
- Trying to put a full `if` statement directly inside `{ }` in JSX (`{ if (x) { ... } }`) — JSX only accepts expressions, so use a ternary, a lookup, or move the `if` logic above the `return` statement instead.
- Mutating props directly inside a component (e.g. `status = "changed"`) — props are read-only; attempting to reassign them doesn't affect the parent and signals a design mistake (you probably want state instead).

Debugging Tips

- React DevTools' "Components" tab lets you click any rendered component and see its exact props in a side panel — extremely useful for confirming data actually arrived the way you expect.
- The console warning about missing `key` props includes the component name and file, letting you jump straight to the `.map()` call that needs fixing.
- If a prop shows up as `undefined` inside a component, double-check the *spelling* on both sides — the prop name passed by the parent (`<CrewCard name={...} />`) must exactly match the name destructured in the child (`{ name }`).
- Use `console.log({ name, callsign, status })` as the first line inside a component's function body to quickly inspect exactly what props it received on a given render.

Check Your Understanding

1. What is the difference between HTML's `class` attribute and JSX's equivalent?
2. Why can't you put a full `if` statement directly inside `{ }` in JSX?
3. What object does a component actually receive as its single argument when props are passed to it?

4. Why does React need a `key` when rendering a list, and what makes a good key?
5. What's the difference between a prop and a piece of state (conceptually, even before Lesson 3)?

Mini Quiz + Answer Key

Q1. In JSX, how do you apply a CSS class to an element? A) `class="box"` B) `className="box"` C) `css="box"` D) `style="box"`

Q2. Given `function Greeting({ name }) { ... }`, how would you render it for a user named "Sam"? A) `<Greeting name="Sam" />` B) `<Greeting>Sam</Greeting>` C) `<Greeting props="Sam" />` D) `Greeting("Sam")`

Q3. Why is using an array index as a list's `key` discouraged when the list can reorder? A) It causes a syntax error B) It can cause React to mismatch state between items after reordering C) Indexes are always negative D) `key` doesn't accept numbers

Q4. What must every value inside `{ }` in JSX be? A) A string only B) An expression that produces a value C) An HTML tag D) A comment

Answer Key: 1-B, 2-A, 3-B, 4-B

Lesson Recap

You learned how JSX embeds real JavaScript expressions with `{ }`, how props pass data from parent to child components (mirroring function arguments), how to destructure props for cleaner code, and how to render dynamic lists safely using `.map()` with a proper `key`.

Homework

Build a `PlanetList` feature: a `planets.js` data file with at least five objects (`name` , `distanceFromSunAU` , `hasRings`), a `PlanetCard` component that takes a `planet` prop and displays its fields (showing "Has rings 🌌" only when `hasRings` is true), and a `PlanetList` component that maps over the data array with proper keys.

Portfolio Project Connection

Project 07's color/size swatches, Project 08's entire feed, and Project 09's project cards are all lists of components rendered from real data arrays using `.map()` and `key` — precisely the pattern from this lesson.